ACL 2026

TreeDiff: AST-Guided Code Generation with Diffusion-based LLMs (DLLMs)

Yiming Zeng[1*] · **Jinghan Cao**[2*] · **Zexin Li**[3] · **Yiming Chen**[4] · **Tao Ren**[5] · **Zhuochun Li**[5] · **Dawei Xiang**[1] · **Xidong Wu**[5] · **Shangqian Gao**[6] · **Tingting Yu**[1]

[1]University of Connecticut · [2]San Francisco State University · [3]UC Riverside · [4]National University of Singapore · [5]University of Pittsburgh · [6]Florida State University · * Equal contribution

🎯 **TreeDiff = first AST-aware masking for diffusion code generation → +13.3% on HumanEval+, narrows gap with autoregressive models.**

💡 Motivation

Diffusion-based LLMs (DLLMs) struggle with **syntactic precision** in code generation. Standard random token masking:

- Ignores syntactic boundaries during denoising
- Fails to capture long-range hierarchical dependencies
- Treats all tokens equally — but they're not!

> 🔑 **Core Insight**
> *"For code, noise should not be purely stochastic — structural priors enable DLLMs to learn program-level dependencies."*

🔧 Method: AST-Guided Masking

TreeDiff parses code into an **Abstract Syntax Tree (AST)** and uses it to guide which tokens to mask — targeting structurally critical nodes.

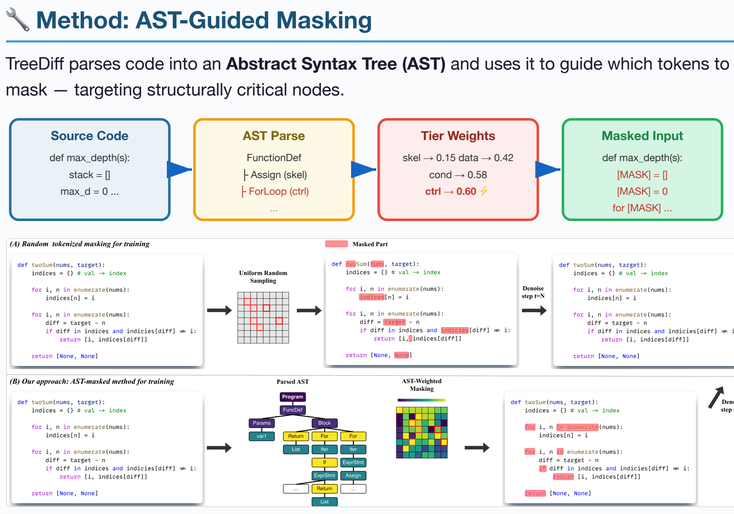


Figure 1: Uniform Random Masking (top) vs. AST-Weighted Sampling Masking (bottom).

⚖️ AST Node Weighting

Category	Examples	Weight	Purpose
Skeletal	Imports, Constants	0.15	Preserve skeleton
Data Flow	Assigns, Calls	0.42	Moderate priority
Conditionals	if, elif, else	0.58	Core logic
Control Flow	for, while, return	**0.60**	⚡ Highest priority

📊 Main Results (Pass@1 %)

13.3%
Relative Gain
HumanEval+ (T=512)

42.1%
HumanEval Pass@1
(vs 39.6% baseline)

Model	T = 256				T = 512			
	HE	HE+	MBPP	MBPP+	HE	HE+	MBPP	MBPP+
Autoregressive Models†								
DeepSeek-Coder-33B	50.6	44.5	80.4	70.1	50.6	44.5	80.4	70.1
CodeQwen1.5-7B	51.8	45.7	73.5	60.8	51.8	45.7	73.5	60.8
CodeLlama-13B	42.7	38.4	63.5	52.6	42.7	38.4	63.5	52.6
Diffusion-based LLMs (LLaDA Backbone)								
LLaDA-Instruct	39.6	34.8	51.9	43.1	36.6	31.7	39.4	31.7
+ Random Masking	39.6	35.4	52.9	43.9	38.4	32.3	41.5	32.8
Ours								
+ TreeDiff	**42.1**	**37.2**	**52.9**	**44.2**	**40.2**	**36.6**	**42.3**	**33.3**

† From EvalPlus Leaderboard. HE = HumanEval, HE+ = HumanEval+.

🧪 Ablation: Masking Granularity

HumanEval		
Strategy	**T=256**	**T=512**
Random Masking	39.6 / 35.4	38.4 / 32.3
AST (Span)	40.9 / 36.6	40.2 / 36.6
AST (Node)	**42.1 / 37.2**	**40.2 / 36.6**

MBPP		
Strategy	**T=256**	**T=512**
Random Masking	52.9 / 43.9	41.5 / 32.8
AST (Span)	51.6 / 42.9	39.7 / 31.5
AST (Node)	**52.9 / 44.9**	**42.3 / 33.3**

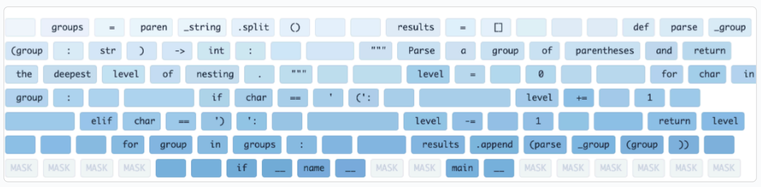
Key finding: Span-level masking corrupts entire subtrees — too coarse. TreeDiff's **token-level** AST masking surgically targets high-influence nodes while preserving context.

🛡️ Robustness Analysis

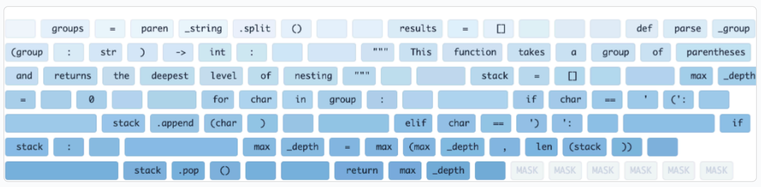
TreeDiff maintains stable performance (−0.1%) as T increases 256→512, while baselines degrade by ~2.1%.

🔍 Generation Trajectory Analysis

Task: Max nesting depth of parentheses (HumanEval/6), Step 110/256.



Random Mask Baseline: Uses shallow counter heuristic, fails on nested structures. Fragmented generation.



TreeDiff: Establishes stack-based control flow (stack.pop()) early. Correctly captures recursive nesting.

💻 Generated Code Comparison

HumanEval/6 — Nested parentheses depth:

✅ **TreeDiff**

```
for char in group:
    if char == '(':
        stack.append(char)
        max_depth = max(
            max_depth, len(stack)
        )
    elif char == ')':
        if stack: stack.pop()
```

❌ **Random Masking**

```
count = 0
for char in group:
    if char == '(':
        count += 1
    # Fails to track
    # nested peak depth
```

🏆 Key Contributions

- **First AST-aware masking** for DLLMs in code generation
- **13.3% relative improvement** on HumanEval+ vs random masking
- Trained on **150K** long code reasoning samples
- Negligible overhead: AST parsing is O(L)

📝 Conclusion

TreeDiff addresses a fundamental limitation: **random masking ignores code structure**. By aligning corruption with AST hierarchy, the model learns programming language compositionality, reconstructing programs that respect grammatical boundaries and capture long-range dependencies.

Impact: Structural priors unlock DLLMs for code — TreeDiff narrows the gap between diffusion and autoregressive paradigms.

Setup: LLaDA-8B backbone · 8× NVIDIA A100 · LoRA (r=64, α=128) · 4096 max tokens · LR 5×10^{-5} · Benchmarks: HumanEval/+, MBPP/+

Yiming Zeng
yiming.zeng@uconn.edu
LinkedIn WeChat

Tao Ren
tar118@pitt.edu
WeChat LinkedIn